조치 가능한 버그 예측을 위한 유사 패치 추천

신지호 ○ 남재창
한동대학교
{21931006, jcnam}@handong.edu

Similar Patch Recommendation for Actionable Defect Prediction

Jiho Shin and Jaechang Nam
Handong Global University

요 약

소프트웨어 품질 보증 과정에서 발생되는 비용을 줄이기 위해 버그 검출/예측, 버그 위치 추정 (Fault Localization), 자동 패치 생성 (Automated Program Repair) 등 다양한 자동화 연구들이 진행되고 있다. 하지만 버그 예측과 같은 연구는 결과에 대한 설명이 어렵고 (explainability) 즉시 조치하기 어려운 문제가 (actionability) 있기 때문에 새로운 방법인 유사 커밋 검색을 통한 유사 패치 추천 기법을 제안하고자 한다. 개발자가 새로 커밋한 코드가 기존의 Bug Introducing Commit(BIC)과 유사하다면 버그 검출/예측 시 즉시 조치할 수 있는 정보를 제공하여 품질 보증에 기여할 수 있을 것으로 기대한다.

1. 서 론

소프트웨어의 복잡도가 지속적으로 늘고 있음에 따라 프로젝트들의 유지/보수 비용도 함께 증가하고 있다. 이와 같은 문제를 해결하기 위해 버그 예측과 같은 품질보증 프로세스를 자동화하는 연구들이 활발히 일어나고 있다. 하지만 기존 버그 예측 분야에서는 예측의 입상(granularity)이 크기 때문에 예측된 결과를 설명하기 어렵고 즉시 조치하지 못한다는 문제점이 있다.

이를 해결하기 위하여 유사 커밋 검색과 패치 커밋 추천 기법을 제안한다. 소프트웨어 프로젝트의 버그 리포트에는 버그에 대한 정보와 이를 수정한 패치 커밋들의 정보들이 축적되어 있다. 패치 정보들을 가지고 SZZ 알고리즘[1]을 적용하면 어느 커밋에 버그가 유입되었는지 알 수 있기 때문에 기존 소프트웨어 저장소에 있는 BIC 들과 현재 개발자가 커밋한 정보가 유사하다면 버그로 인식함과 동시에 패치를 제안할 수 있게 된다.

현재 이와 연관된 연구들로는 코드 검색, 코드 클론 (clone) 검출 및 검색, 커밋 군집, 자동 패치 생성 등이 있지만 코드 검색 같은 경우 아주 짧은 코드를 입력 받거나 자연어를 입력 받는 경우만 작동이 가능하고 코드 클론 검출/검색은 코드의 변화를 나타내는 커밋과는 형태의 차이가 분명하다. 또한, 자동 패치 생성과 같은 경우 처리할 수 있는 버그의 형태가 한정적이고 생성된 패치들이 테스트 케이스에 과다 적합 (overfitting) 되는 성향이 있다.

따라서 본 논문은 코드의 수정 정보를 갖고 있는 커밋을 소프트웨어 저장소에 버그라고 레이블 되어 있는 커밋들과 유사도를 비교하고 비슷한 커밋 일 경우, 버그 커밋을 고친 패치 커밋을 추천하는 아이디어를 제안한다.

2. 본 론

유사한 코드 커밋 검색의 가능성을 검증하기 위해 1) 수정된 코드를 벡터화 시켜 비교하는 실험과 2) 문자열로써 비교하는 실험을 실행하였고, 실험 결과를 바탕으로 3) 딥러닝으로 추출한 의미적 특징으로 유사도를 측정하는 방법을 제안한다.

3.1. 수정-벡터 수집 (Change Vector Collector)

세밀한 단위로 코드의 수정 내역을 이해하기 위해 Gumtree [2]라는 트리 비교 알고리즘을 사용했다. Gumtree 를 이용하면 수정된 양의 단위를 AST 노드 단위로 줄이고 추가된 코드와 제거된 코드 뿐만이 아닌 수정된 코드와 이동한 코드도 찾을 수 있다 [2]. 따라서 네 가지 (추가, 삭제, 수정, 이동) 작업에 대한 AST 노드의 수로 벡터를 추출하였다.

이 추출 방법으로 깃 허브에 있는 버그 수정에 대한 커밋들의 수정-벡터를 추출하고 확률 밀도 함수에서 널리 쓰이는 알고리즘 중 피어슨 상관 계수와 코사인 유사도를 사용했고 변수들의 순위 사이의 유사도를 검사하는 스피어만 상관 계수와 켄달 상관 계수를 적용하여 유사도를 측정하였다. 마지막으로 벡터간의 유사도를 집합관계에서 보여주는 자카드 지수를 사용하여 계산하였다.

그림 1.1 과 1.2 는 수정-벡터를 뽑은 후 자카드 지수를 취했을 때에 0.92 가 나온 예시이다. 그림 1.3 은 해당 커밋들에 대한 수정-벡터이다. 그림 1 의 예제와 같이 ImportDeclaration 을 추가한 인덱스 번호가 146 에 해당되고 MethodInvocation 은 159 에 해당된다. 이와 같이 수정된 AST 노드를 카운트하는 식으로 벡터를 이루었다.

하지만 이러한 코드들의 수정이 다른 메서드나 매우 다른 문맥에서 나오더라도 같은 AST 노드가 바뀌었다면 같은 카운트로 표기가 된다. 따라서 다른 코드의 위치로 인해 의미가 전혀 다르더라도 벡터가 비슷하게 나오고 그에 따른 상관계수도 높게 나오는 현상이 있었다.

```
+ import
org.apache.isis.viewer.wicket.ui.components.property.PropertyEditFor
mExecutor;
+         scalarModel.setFormExecutor(new
PropertyEditFormExecutor(scalarModel));
```

그림 1.1 수정-벡터 Isis 인스턴스 결과 예시

```
+ import
org.apache.ignite.internal.processors.rest.handlers.redis.key.GridRe
disExpireCommandHandler;
+         addCommandHandler(new GridRedisExpireCommandHandler(log,
hnd));
```

그림 1.2 수정-벡터 Ignite 인스턴스 결과 예시

{127 1,141 1,146 1,159 1,177 1,181 4,182 1,211 1,212 1}
{127 1,141 1,146 1,159 1,177 1,181 4,182 1,211 0,212 1}
그림 1.3 수정-벡터 예시

※ 본 연구는 2019 년도 정부(과학기술정보통신부)의 재원으로 한국연구재단의 지원을 받아 수행된 연구임(No. 2018R1C1B6001919).

3.2. 문자열 비교

커밋 코드들의 순서/문맥 정보를 보존하기 위해 코드 커밋을 문자열의 형태로 표현하고 문자열들의 유사도를 계산하는 알고리즘을 적용하였다.

먼저 커밋의 문자열들을 추출하기 위해서 검트리를 사용하지 않고 수정된 라인 전체를 취하는 기존의 비교 방법을 (git diff) 사용하였다. 검트리를 사용하면 바뀐 부분의 AST 노드 만을 취하기 때문에 연속된 코드의 정보를 얻지 못하기 때문이다.

유사도를 측정하기 전에 노이즈 정보를 최소화 하기 위해 전처리 과정을 다음과 같이 실시하였다. 먼저 같은 메서드 범위에서 제거된 라인과 (버그) 추가된 라인이 (패치) 모두 있어야 버그 수정과 관련된 수정이라 판단하고 제거나 추가만 있는 라인들은 모두 무시한다. 또한 코스메틱 수정에 불과한 공백들도 모두 제거한다.

문자열들의 유사도 검증 알고리즘은 벡터 계산보다 시간이 오래 걸리기 때문에 커밋 코드에 SimHash 알고리즘을 적용하여 코드의 정보를 잃는 것을 최소한으로 차원을 축소하고 일정 이상의 해밍 거리 (Hamming distance) 보다 멀면 너무 다른 문자열이라 판단하여 비교를 건너뛰는 방식을 사용하였다 [3].

문자열 유사도 비교 알고리즘으로는 자카드 거리와 레벤슈타인 거리를 사용하였다. 자카드 거리는 두 문자열을 집합으로 보고 비율적으로 얼마나 포함하는지를 측정하는 알고리즘이다. 레벤슈타인 거리는 두 문자열의 수정되는 횟수로 거리를 측정하는 알고리즘이기 때문에 순서를 고려할 수 있다. 따라서 이 두 알고리즘의 조합을 사용하였다.

```
private XMLInputSource getInputSource() throws IOException {
  try {
    return new XMLInputSource(aePath);
  } catch (IOException e) {
    return new XMLInputSource(getClass().getResource(aePath));
  }
}
```

그림 2.1 문자열 비교 Lucene-solr 예시 (제거된 라인)

```
private XMLInputSource getInputSource() throws IOException {
  try {
    return new XMLInputSource(aePath);
  } catch (Exception e) {
    return new XMLInputSource(getClass().getResource(aePath));
  }
}
```

그림 2.2 문자열 비교 Lucene-solr 예시 (추가된 라인

```
        LOG.warn("Request remote address is NULL");
        hostname = "???";
    }
} catch (Exception ex) {
    LOG.warn("Request remote address could not be resolved,
```

그림 2.3 문자열 비교 Oozie 예시 (제거된 라인)

```
        LOG.warn("Request remote address is NULL");
        hostname = "???";
    }
} catch (UnknownHostException ex) {
    LOG.warn("Request remote address could not be resolved,
```

그림 2.4 문자열 비교 Oozie 예시 (추가된 라인)

3.2.2. 결과

그림 2 는 코드 커밋을 문자열로서 비교했을 때 상관계수가 0.86 이 나온 예시이다. 결과와 같이 수정의 정도가 매우 짧은 인스턴스들만이 높은 상관계수를 보였다. 더 긴 예시들도 있었지만 상관계수가 낮기 때문에 실용성이 떨어지는 것을 볼 수 있었다.

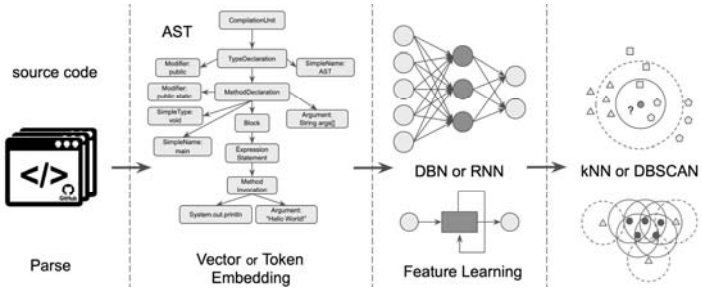


그림 3 딥러닝을 이용한 방법의 시스템 개요도

3.3. 딥러닝을 활용한 유사 커밋 검색

3.1 과 3.2 의 방법은 AST 노드 출현 횟수나 문자열의 유사도와 같이 사람이 직접 특징들을 찾아야 하는 방법이지만 의미적인 특징들 없이 문자적 특징들만 갖고 있다는 단점이 있다. 따라서 의미적인 특징을 컴퓨터가 직접 뽑을 수 있도록 딥러닝 방법을 적용하여 커밋들을 분류하는 방법을 제안한다.

그림 3 은 딥러닝을 활용한 유사 커밋 검색 시스템에 대한 개요도를 보여준다. 의미적 특징을 뽑기 위해 커밋 코드를 고차원 벡터 공간에 임베딩하고 DBN(Deep Belief Network)이나 RNN 같은 딥러닝 알고리즘을 적용하여 각 인스턴스의 특징들을 뽑아 낸다. 이러한 방법으로 코드 뭉치의 의미적 특징을 뽑아내는 연구로 [4, 5]와 같은 연구들이 있었지만 이 연구들은 버그 예측 분야에서 활용하였기 때문에 최종적으로 이진 분류가 (버그/클린) 목적이다. 딥러닝 모델을 사용한 버그 예측 분류기들이 기존 파일 단위 버그 예측 연구와 JIT 버그 예측 연구에서 성능을 크게 향상시켰다. 따라서 딥러닝을 활용하여 의미적 특징들을 뽑아 내는 것이 기존의 사람들이 뽑은 특징보다 더 효과적일 수 있다. 이런 연구들처럼 딥러닝으로 특징들을 뽑아내고 kNN(k-Nearest Neighbor)이나 DBSCAN 과 같은 알고리즘을 사용하여 이진 분류가 아닌 특징적으로 거리가 가까운 유사 커밋을 찾아 내는 것이 목표이다.

[참고 문헌]

[1] Kim, Sunghun, et al. "Automatic identification of bug-introducing changes." 21st IEEE/ACM International Conference on Automated Software Engineering (ASE'06). IEEE, 2006.

[2] Falleri, Jean-Rémy, et al. "Fine-grained and accurate source code differencing." Proceedings of the 29th ACM/IEEE international conference on Automated software engineering. ACM, 2014.

[3] Uddin, Md Sharif, et al. "On the effectiveness of simhash for detecting near-miss clones in large scale software systems." 2011 18th Working Conference on Reverse Engineering. IEEE, 2011.

[3] Wang, Song, Taiyue Liu, and Lin Tan. "Automatically learning semantic features for defect prediction." 2016 IEEE/ACM 38th International Conference on Software Engineering (ICSE). IEEE, 2016.

[4] Yang, Xinli, et al. "Deep learning for just-in-time defect prediction." 2015 IEEE International Conference on Software Quality, Reliability and Security. IEEE, 2015.